Plant Whisperer

An IoT, Deep Neural Network and Generative AI system for smart plant care | Project Report

1. Abstract

Houseplants are often over-watered or under-watered because the signs of stress appear late and raw sensor numbers mean little to most people. This project presents **Plant Whisperer**, a low-cost system that combines three technologies. An ESP32 microcontroller with a soil moisture sensor, a DHT11 temperature and humidity sensor and a light-dependent resistor (LDR) collects environmental data and sends it over WiFi to a server. A small Deep Neural Network (DNN) built with Keras uses these readings to predict the number of hours until the soil becomes dry. A Generative AI model (Claude) then converts the sensor facts and the prediction into a short, friendly message written in the plant's own voice, which is delivered through a Telegram bot and a web dashboard.

The complete software pipeline, covering the firmware, the trained model, the server, the language model integration, the dashboard and a hardware simulator, has been built and tested. The DNN is trained on synthetic data generated from a drying model and reaches a validation mean absolute error of about 0.7 hours on that data. Hardware assembly and calibration with real sensors are the next stage. The estimated hardware cost is about ₹830.

2. Introduction

The Internet of Things (IoT) connects small, inexpensive devices to the internet so that they can measure the physical world and report what they sense. Deep learning allows a computer to learn relationships between many measurements, and Generative AI allows a computer to explain information in natural language. Each technology is useful alone, but together they can turn raw measurements into clear, actionable advice.

Plant care is a practical place to apply this combination. Whether a plant needs water depends not only on how dry the soil is now, but also on temperature, humidity and light, because these control how quickly moisture is lost. A person looking at a single moisture number cannot easily judge how soon the plant will need attention.

Plant Whisperer is designed in three layers, each with one clear job:

- **IoT layer:** collects soil moisture, temperature, humidity and light readings.
- **DNN layer:** predicts the hours until the soil reaches the dry threshold.
- **GenAI layer:** explains the situation in plain language, in the voice of the plant.

In short, the DNN decides what is happening and the GenAI explains it.

3. Problem Statement

Plant owners frequently lose plants because watering is done by habit or guesswork. The main problems are:

1. Signs of over-watering and under-watering become visible only after damage has started.

2. A raw soil moisture value, such as "2150", is not meaningful to a non-technical user.
3. Fixed threshold alerts ignore the environment. The same moisture level can mean a few hours of safety on a cool, humid day and a few minutes of safety on a hot, bright day.
4. Most simple monitors report the present state only. They do not predict when action will be needed.

The problem addressed by this project is therefore: *to design a low-cost system that measures a plant's environment, predicts when the plant will need water, and communicates this to the user in a clear and engaging way.*

4. Existing System

Current approaches to plant care and monitoring include the following:

- **Manual watering:** the owner waters on a fixed schedule or by touching the soil. This depends on experience and memory and ignores weather conditions.
- **Timer-based watering:** simple timers water at fixed intervals regardless of the plant's actual need.
- **Threshold-based moisture alarms:** a soil sensor raises an alert when moisture falls below a fixed value. These react only after the soil is already dry, and they do not account for temperature, humidity or light.
- **Commercial smart sensors and apps:** some products report moisture and other readings to a phone app. They are typically closed systems, can be expensive, and mostly present numbers and charts rather than predictions explained in plain language.

The common limitations are that these systems are reactive instead of predictive, use fixed rules, present data in a technical form, and cannot be easily extended by the user.

5. Proposed System

The proposed system, Plant Whisperer, is an open and low-cost pipeline that adds prediction and natural language explanation to basic sensing.

- An **ESP32** reads a soil moisture sensor, a DHT11 and an LDR, and sends the readings to a server every 30 seconds.
- A **Flask server** converts the raw values into percentages, computes the recent drying rate, and runs a **DNN** that predicts the hours until the soil reaches the 25% dry threshold.
- The server assigns a mood (thirsty, happy or drowning) from the soil moisture level.
- A **Generative AI model** receives only the measured facts and the prediction and writes a short in-character message. The message is sent through **Telegram** only when the mood changes, to avoid spamming the user.
- A **web dashboard** shows the live mood, readings, a history chart and a chat box, so the user can ask the plant how it feels.
- A **hardware simulator** sends realistic readings to the same server, so the whole software pipeline can be tested and demonstrated before the sensors are assembled.

The key difference from existing systems is that the proposed system predicts ahead using several environmental factors and explains the result in plain language.

6. System Architecture

The data flows through five stages:

Sensors → ESP32 → Flask server with DNN → Generative AI (Claude) → Telegram bot and web dashboard

1. **Sensors:** the soil moisture sensor (analog) is connected to GPIO34, the DHT11 (digital) to GPIO4, and the LDR with a 10k resistor forming a voltage divider to GPIO35. All components work at 3.3 V.
2. **ESP32:** the microcontroller averages ten readings of the analog sensors to reduce noise, reads temperature and humidity, and sends the values as a JSON message to the server using an HTTP POST every 30 seconds.
3. **Server and DNN:** the server converts the raw values into percentages using calibration constants, estimates the drying rate from the last two hours of readings, normalizes the five inputs, and runs the trained network.
4. **Generative AI layer:** the sensor facts, mood and predicted hours are passed to Claude with instructions to act as a witty plant and never to invent numbers. If the model cannot be reached, the server uses built-in message templates so the system keeps working.
5. **Output:** messages go to Telegram when the mood changes. The dashboard reads the latest values from the server every 2 seconds and supports a chat endpoint for questions to the plant. Every reading is also saved in a CSV log for later fine-tuning.

The heavy computation (the DNN and the language model) runs on the server side, which keeps the microcontroller simple, cheap and reliable.

7. Hardware and Software Requirements

7.1 Hardware requirements

Component	Purpose	Approx. cost (INR)
ESP32 development board	WiFi-enabled microcontroller	450
Soil moisture sensor	Measures soil dryness (analog)	80
DHT11	Temperature and humidity	120
LDR with 10k resistor	Light level	30
Breadboard and jumper wires	Connections, no soldering needed	150
Total		**about 830**

Prices are rough estimates and vary by seller. A laptop or computer is also needed to run the server, and a USB cable to program the ESP32.

7.2 Software requirements

Software	Purpose
Arduino IDE with ESP32 support	Programming the ESP32 firmware
DHT sensor library and Adafruit Unified Sensor	Reading the DHT11
Python 3	Server, training and simulator scripts
Flask	Web server and API endpoints
TensorFlow and Keras	Building, training and running the DNN
NumPy	Numerical processing and data generation
Anthropic Python SDK	Calling the Generative AI model
Requests	Sending Telegram messages and simulator readings
Telegram Bot API (via BotFather)	Delivering messages to the user's phone
Web browser	Viewing the dashboard

Google Colab can optionally be used for training. Telegram, Colab and the open-source libraries are free. The language model API is billed by usage, which is small for a demonstration because the plant speaks only when its mood changes.

8. Methodology

The project was carried out in the following steps.

8.1 Data acquisition

The ESP32 firmware connects to WiFi, then every 30 seconds it takes ten readings from each analog pin and averages them, reads the DHT11, skips the cycle if the DHT11 read fails, and posts a JSON message containing the raw soil value, raw light value, temperature and humidity.

8.2 Pre-processing and calibration

The server converts raw analog values into percentages using two calibration points for each sensor, the value in dry air and the value in water (and for the LDR, dark and bright). The calibration constants must be measured for each physical sensor. The server also estimates the **drying rate** (percentage of moisture lost per hour) from the last two hours of readings. If there are not enough readings, a default of 0.5% per hour is used. When the soil moisture rises sharply, which indicates watering, the history is cleared so that the trend restarts.

8.3 Deep Neural Network

The DNN predicts the hours until the soil falls to the 25% dry threshold.

- **Inputs (5):** soil moisture %, temperature, humidity, light %, and drying rate.
- **Architecture:** Dense layer with 32 units, Dense layer with 16 units (both ReLU), and a single output unit.

- **Training setup:** Adam optimizer, mean squared error loss, 30 epochs, batch size 128, with 15% of the data held out for validation. The inputs are normalized using the training mean and standard deviation, and the same values are applied to live data on the server.

Because a long period of real data was not available, the network was trained on **40,000 synthetic samples**. Soil moisture, temperature, humidity and light were sampled at random, and a simple physical-style drying model gave the true rate:

rate = 0.25 + 0.07 × (temperature − 20) + 0.02 × light − 0.015 × (humidity − 50), with a minimum of 0.08

Heat and light dry the soil faster, and humidity slows it down. The target was the hours until the soil reaches 25%, capped at 72 hours. The drying rate fed to the network was the true rate with 15% random noise, to imitate a noisy sensor trend.

8.4 Mood classification

The mood is decided from the soil moisture percentage: below 25% is **thirsty**, above 80% is **drowning** (too wet), and anything in between is **happy**.

8.5 Generative AI message generation

The measured facts and the DNN prediction are placed in a prompt, and a system prompt instructs the model to behave as a witty, slightly dramatic houseplant, to keep replies brief, and to base everything strictly on the given facts without inventing numbers. A message is produced only when the mood changes. The same mechanism answers user questions through a chat endpoint. If no API key is available or the call fails, built-in templates are used instead.

8.6 Delivery, logging and testing

Messages are sent through a Telegram bot. The web dashboard polls the server every 2 seconds and shows the mood, readings, predicted time until dry, a soil history chart and a chat box. Each reading is appended to a CSV file. To test without hardware, a **simulator** acts as the ESP32 and streams a realistic scenario: the plant dries out, becomes thirsty, is watered, becomes too wet, and dries back to normal.

9. Result

The results below reflect the current stage of the project: the software pipeline is complete and was tested with the simulator. Measurements from real sensors will be added after the hardware is assembled.

9.1 Model training

Measure	Value
Training samples	40,000 (synthetic)
Validation split	15%
Epochs	30
Validation mean absolute error	about 0.72 hours
Validation mean squared error	about 2.29 hours squared

These figures show that the network learned the assumed drying relationship well. They do **not** measure accuracy on a real plant, because the training data is synthetic.

9.2 Model behaviour

The table shows the hours-to-dry values given by the drying formula for a plant at 60% soil moisture, 50% light and 50% humidity. These are the relationships the network is trained to reproduce.

Temperature	15°C	20°C	25°C	30°C	35°C	40°C
Hours to dry	38.9	28.0	21.9	17.9	15.2	13.2

The hotter the environment, the sooner the plant will need water, which is the effect a fixed threshold cannot capture.

9.3 End-to-end software test

The server and simulator were run together. The following behaviour was observed:

- The server returned "no data yet" before the first reading and returned the full set of values after the simulator started.
- The mood moved through happy, thirsty, drowning and back to happy as the simulated plant dried, was watered and dried again.
- The predicted hours to dry fell as the soil dried and rose after watering.
- Messages were generated only when the mood changed, and the chat endpoint replied with a message based on the latest readings.
- The dashboard page was served by the server and received the latest readings and history.

9.4 Hardware results

Not yet obtained. Sensor assembly, calibration and a real-data test will follow, and the logged readings will be used to fine-tune the network.

10. Advantages and Limitations

Advantages

- **Predictive, not just reactive:** it estimates when water will be needed instead of alerting only when the soil is already dry.
- **Uses several environmental factors,** so predictions respond to heat, humidity and light.
- **Easy to understand:** messages are in plain language and in a friendly voice.
- **Low cost:** about ₹830 in hardware, no soldering and no custom circuit board.
- **Lightweight device:** the heavy processing is done on the server, so the ESP32 stays simple.
- **Reliable demonstration:** the simulator and fallback messages allow testing and demos without hardware or internet access to the language model.
- **Extensible:** the logged data can be used to retrain the model, and new sensors can be added.

Limitations

- **Synthetic training data:** the model currently learns an assumed formula, not real plant behaviour. Its real-world accuracy is not yet measured.
- **Sensor limitations:** the DHT11 has limited accuracy, the ESP32 analog input is not perfectly linear, and low-cost resistive soil sensors corrode over time. Calibration is required.
- **Dependence on a server and network:** a computer must be running and reachable over WiFi, and the language model needs internet access and an API key.
- **Plant-specific behaviour is ignored:** different species and pot sizes dry at different rates, and the model currently treats all plants alike.
- **Trend resets after watering:** right after watering, the drying rate estimate takes time to rebuild.
- **Hardware not yet validated:** results from real sensors are pending.

11. Application

- **Home gardening:** apartment and balcony plants, especially for busy owners or people who travel.
- **Education:** a hands-on project that teaches IoT, machine learning and generative AI together, suitable for schools and colleges.
- **Nurseries and greenhouses:** monitoring many plants with predictions of which need water first.
- **Offices and public spaces:** keeping indoor plants healthy with fewer manual checks.
- **Small-scale agriculture and research:** monitoring soil and weather conditions for crops or experiments.
- **Accessibility:** natural language messages make plant data understandable for people who find numbers and charts hard to interpret.

12. Future Scope

- **Fine-tune on real data:** use the readings logged from the real sensors to retrain the network so that it reflects actual plant behaviour.
- **Time-series models:** replace the simple network with an LSTM that learns from sequences of readings.
- **Automatic watering:** add a water pump or valve controlled by the prediction.
- **Multi-plant dashboard:** support many plants, each with its own sensors and species profile.
- **Species-aware predictions:** include plant type, pot size and soil type as inputs.
- **On-device inference:** run a compressed model on the ESP32 itself using TinyML so that it works without a server.
- **Better sensors:** use capacitive soil sensors that do not corrode, and a more accurate temperature and humidity sensor.
- **Native mobile app and voice:** a dedicated phone app, push notifications and voice interaction with the plant.

13. Conclusion

Plant Whisperer shows that IoT, deep learning and generative AI can work together as three layers with distinct roles: the sensors measure, the neural network predicts, and the language model explains. The project turns raw soil, temperature, humidity and light readings into a clear prediction of when a plant will need water and delivers it as a friendly message through Telegram and a web dashboard.

The software pipeline is complete and was verified end to end with a hardware simulator, and the estimated hardware cost of about ₹830 keeps the project accessible to students and hobbyists. The main limitation is that the neural network is currently trained on synthetic data, so the next steps are to assemble the sensors, calibrate them, and fine-tune the model on real readings. With these steps, the system can grow from a working prototype into a practical tool for smarter plant care.